

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

COMPUTACIÓN GRÁFICA e INTERACCIÓN

HUMANO COMPUTADORA



EXAMEN EXTRAORDINARIO DE LABORATORIO DOCUMENTACIÓN TÉCNICA

NOMBRE COMPLETO: Ceron Soto Edwin Rodrigo

Nº de Cuenta: 420052964

ASIGNATURA: Computación Gráfica e Interacción Humano Computadora

EXAMEN: Extraordinario 2027-1 – Escena de la Revolución Mexicana

SEMESTRE 2027-1

FECHA DE ENTREGA: 30/10/2026

Instrucciones:

La temática general del proyecto es la Revolución Mexicana. Se debía integrar y acomodar de forma coherente la escena para que fuera alusiva al tema, realizando las siguientes actividades:

1. Cambiar la luz direccional a blanco tenue y agregar una tecla para alternarla entre blanco y naranja, con una intensidad de 0.5.

2. Integrar el modelo del zorro (dividido en varias partes) y crear dos animaciones por código, caminar y saltar, asociadas a las teclas Z y X.

3. Integrar 15 modelos 3D relacionados con la temática, documentar su origen (formato APA) y crear animaciones: en bucle, con pausa y reanudación (teclas C y V), procedurales y por huesos.

4. Integrar luces tipo spotlight de colores sobre uno de los personajes, con encendido y apagado independiente por teclado.

5. Ajustar las UVs y texturas del modelo del piso.

6. Acomodar correctamente los árboles existentes y agregar 4 árboles más.

Desarrollo de actividades:

El proyecto se desarrolló en etapas. Primero se cambió la luz direccional (de roja a blanca y naranja con la tecla L); después se corrigió la iluminación de los objetos para que no se vieran afectados por el cambio de color; enseguida se agregaron los modelos de la escena, luego las animaciones y, por último, las luces roja y azul sobre Steve.

1.- Modificación de la iluminación

El proyecto base emitía una luz direccional de color rojo (Figura 1, primera ejecución). Se cambió a una luz blanca tenue con intensidad de 0.5 y se agregó la variable *luzNaranja*, que se alterna con la tecla L para cambiar el color de la luz entre blanco y naranja (1.0, 0.5, 0.0). Las componentes ambiental, difusa y especular se calculan a partir del color y de la intensidad, por lo que el cambio afecta a toda la escena:

```
glm::vec3 colorLuz(1.0f, 1.0f, 1.0f);
// Cambio de luz con L.
if (luzNaranja) colorLuz = glm::vec3(1.0f, 0.5f, 0.0f);
float intensidadLuz = 0.5f;

// Ajuste de iluminación de los objetos.
glm::vec3 ambient = colorLuz * intensidadLuz * 0.4f;
glm::vec3 diffuse = colorLuz * intensidadLuz;
glm::vec3 specular = colorLuz * intensidadLuz * 0.3f;
```

```
// Cambio de luz direccional con L.
if (key == GLFW_KEY_L && action == GLFW_PRESS)
{
    luzNaranja = !luzNaranja;
    std::cout << "Luz direccional: " << (luzNaranja ? "naranja" : "blanca") << std::endl;
}
```



Figura 1. Primera ejecución: luz direccional roja original del proyecto base.

Después de cambiar el color se ajustó el cálculo de iluminación de los objetos para que no se vieran afectados de forma indebida por la nueva luz y conservaran su aspecto (Figura 2).



Figura 2. Luz direccional blanca tenue con los objetos ya corregidos y el zorro integrado.

2.- Integración y animación del zorro

El zorro se cargó desde la carpeta Models en seis piezas: cuerpo, cabeza, cola y cuatro patas. El cuerpo es la raíz de la jerarquía y cada parte gira alrededor de su articulación respecto al

cuerpo. La altura del zorro se calcula sobre el piso en cada cuadro, por lo que no flota ni se hunde. Se ubicó en una zona despejada, en (5, 0, 0), para que sus animaciones no choquen con otros objetos.

```
auto dibujarParteZorro = [&](Model& parte, const glm::vec3& punto,
    float angulo, const glm::vec3& eje)
    {
        glm::mat4 parteModelo = glm::translate(modeloZorro, punto);
        parteModelo = glm::rotate(parteModelo, glm::radians(angulo), eje);
        parteModelo = glm::translate(parteModelo, -punto);
        glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(parteModelo));
        parte.Draw(lightingShader);
// ...
dibujarParteZorro(zorroCabeza, { 0, -.1f, -.65f }, zorroAnguloCabeza, { 0, 1, 0 });
dibujarParteZorro(zorroCola, { 0, 0, -1.8f }, zorroAnguloCola, { 0, 1, 0 });
dibujarParteZorro(zorroPataID, { .267f, -.493f, -.918f }, zorroAnguloPatas, { 1, 0, 0 });
dibujarParteZorro(zorroPataDD, { -.254f, -.493f, -.918f }, -zorroAnguloPatas, { 1, 0, 0 });
dibujarParteZorro(zorroPataIT, { .267f, -.493f, -1.518f }, -zorroAnguloPatas, { 1, 0, 0 });
dibujarParteZorro(zorroPataDT, { -.275f, -.493f, -1.518f }, zorroAnguloPatas, { 1, 0, 0 });
```

Caminar (tecla Z): la tecla Z inicia o detiene la caminata. El zorro avanza dos unidades sobre el eje Z, da media vuelta (180°) y regresa. Las patas oscilan en oposición (diagonales), y la cola y la cabeza se mueven con funciones seno.

```
if (zorroCaminando)
{
    zorroTiempoPasos += dt;
    zorroRecorrido = std::fmod(zorroRecorrido + zorroVelocidad * dt,
        zorroDistancia * 2.0f);
    if (zorroRecorrido < zorroDistancia)
    {
        zorroPosZ = zorroInicioZ + zorroRecorrido;
        zorroGiro = 0.0f;
    }
    else
    {
        zorroPosZ = zorroInicioZ + zorroDistancia * 2.0f - zorroRecorrido;
        zorroGiro = 180.0f;
    }
    zorroAnguloPatas = 25.0f * std::sin(zorroTiempoPasos * 8.0f);
    zorroAnguloCola = 12.0f * std::sin(zorroTiempoPasos * 4.0f);
    zorroAnguloCabeza = 3.0f * std::sin(zorroTiempoPasos * 4.0f);
}
```

Saltar (tecla X): la elevación sigue una parábola, $4 \cdot h \cdot t \cdot (1-t)$, por lo que el zorro sube y baja suavemente hasta regresar al piso en 0.8 segundos. Durante el salto se flexionan las patas y se levanta la cola. La tecla X no se repite mientras el zorro todavía está saltando.

```
if (zorroSaltando)
{
    zorroTiempoSalto += dt;
    float avanceSalto = zorroTiempoSalto / zorroDuracionSalto;
    if (avanceSalto >= 1.0f)
    {
        zorroSaltando = false;
        zorroTiempoSalto = 0.0f;
        zorroElevacion = 0.0f;
    }
    else
    {
        zorroElevacion = 4.0f * zorroAlturaSalto * avanceSalto * (1.0f - avanceSalto);
        zorroAnguloPatas = -20.0f * std::sin(3.14159265f * avanceSalto);
        zorroAnguloCola = 10.0f * std::sin(3.14159265f * avanceSalto);
    }
    return;
}
```

3.- Integración de modelos adicionales y animaciones

Se integraron 15 modelos diferentes relacionados con la Revolución Mexicana. Cada uno se carga con la clase *ObjetoEscenario*, que lee el modelo con Assimp, carga sus texturas, lo escala a una altura definida (para mantener la proporción con Steve) y lo apoya sobre el piso. El hacha de los habitantes y de Steve se construye por código con cubos.

#	Modelo	Archivo	Origen
1	Casa	Casa.fbx	Elaboración propia (Blender)
2	Iglesia	Iglesia.fbx	Elaboración propia (Blender)
3	Barril	barril.fbx	Elaboración propia (Blender)
4	Bandera de México	bandera.fbx	Elaboración propia (Blender)
5	Cactus	cactus.fbx	Elaboración propia (Blender)
6	Aspas del molino	Aspas.fbx	Elaboración propia (Blender)
7	Molino	Molino.fbx	Elaboración propia (Blender)
8	Pozo	Pozo.fbx	Elaboración propia (Blender)
9	Fogata	Fogata.fbx	Elaboración propia (Blender)
10	Sombrero	Sombrero.fbx	Elaboración propia (Blender)
11	Cerca (valla)	Cerca.fbx	Elaboración propia (Blender)
12	Persona (habitante)	Persona.fbx	Elaboración propia (Blender)
13	Oveja	Oveja.fbx	Sketchfab: https://skfb.ly/6QUXD
14	Cabra	Goat.fbx	Sketchfab: https://skfb.ly/oxO7Z
15	Caballo	Caballo.fbx	Sketchfab: https://skfb.ly/6RxOz

Además de estos modelos, la escena incluye los modelos del proyecto base: el piso, el árbol, el zorro, Steve y el Creeper.

Modelos creados en Blender. A continuación se muestran capturas del proceso de elaboración de algunos modelos propios. La casa se construyó con piezas separadas (losa, paredes, piso, puerta y dos ventanas); la iglesia con paredes, techo, puerta, chimenea y cruz, y se le aplicó la textura *adobe.png*; el barril con tapas, laterales y anillos, y se le aplicó la textura *madera.png*; la bandera con tres piezas (verde, blanco y rojo), y en la pieza blanca se aplicó la imagen del escudo (*Escudo.jpg*).

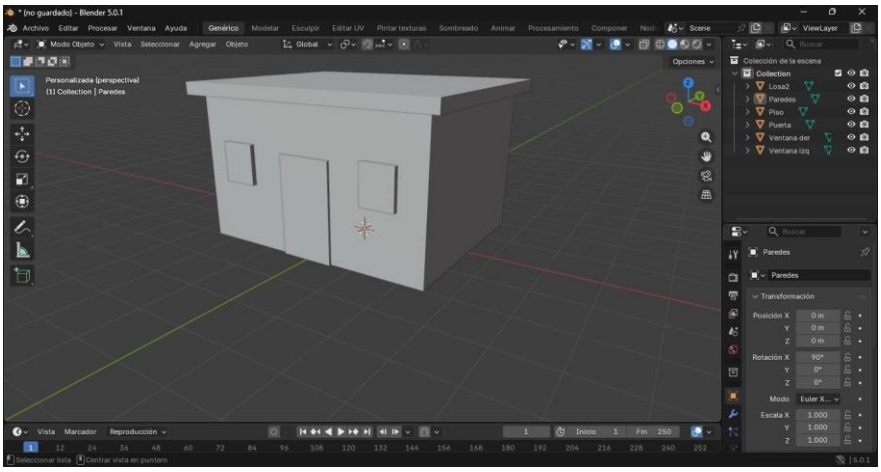


Figura 3. Modelado de la casa en Blender.

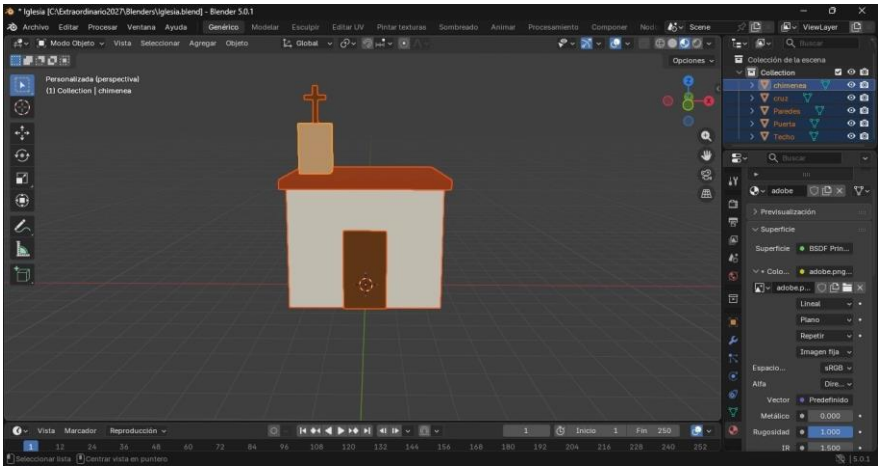


Figura 4. Modelado y texturizado de la iglesia en Blender.

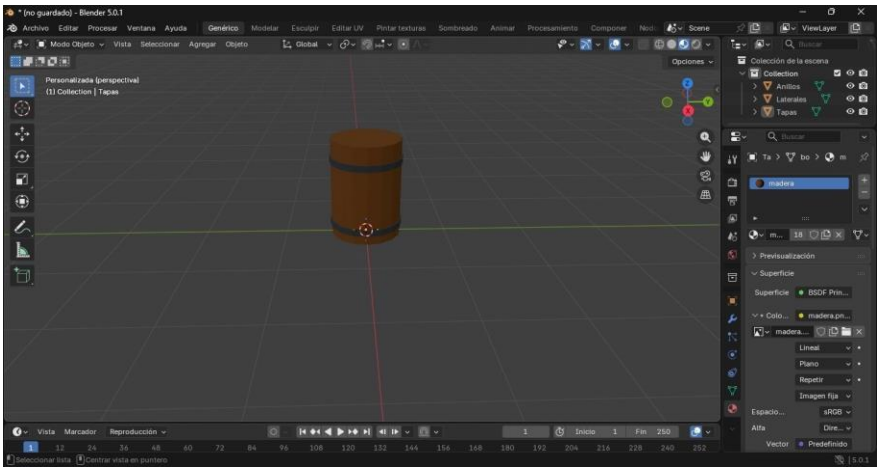


Figura 5. Modelado y texturizado del barril en Blender.



Figura 6. Modelado de la bandera de México en Blender.

Animaciones. Se implementaron las siguientes animaciones, todas controladas desde el código:

Animación	Tecla	Tipo	Descripción
Aspas del molino	Automática	Bucle continuo (procedural)	Las aspas giran de forma continua sobre el eje del molino.
Creeper camina	Automática; C pausa y V reanuda	Bucle con pausa	Camina entre dos puntos, da media vuelta y repite. Se puede pausar y reanudar.
Creeper explota	E	Procedural	Crece, gira y vibra con funciones seno; desaparece y regresa a su lugar de origen.
Steve camina	B	Por huesos (esqueleto)	Reproduce la animación esquelética, avanza y regresa. El hacha y el sombrero siguen a los huesos.
Oveja come	O	Por código	Baja la cabeza al piso, mastica y la vuelve a subir.

Animación	Tecla	Tipo	Descripción
Caballo relincha	H	Por código	Se inclina sobre las patas traseras y regresa.
Hacha de habitantes	G	Por código	Los habitantes levantan y golpean con el hacha tres veces.
Zorro	Z y X	Por código	Caminar y saltar (ver actividad 2).

La animación del Creeper usa las variables *pausa*, *explota* y *creeperCaminando*. Las teclas C y V solo pausan o reanudan la caminata (no actúan durante la explosión):

```
if (key == GLFW_KEY_C && action == GLFW_PRESS)
{
    if (!explota)
    {
        pausa = true;
        std::cout << "Creeper: caminata pausada" << std::endl;
    }
}
if (key == GLFW_KEY_V && action == GLFW_PRESS)
{
    if (!explota)
    {
        pausa = false;
        creeperCaminando = true;
        std::cout << "Creeper: caminata reanudada" << std::endl;
    }
}
```

Las aspas del molino giran continuamente: en cada cuadro se incrementa el ángulo y se aplica al dibujarlas:

```
anguloAspas = std::fmod(anguloAspas + velocidadAspas * deltaTime, 360.0f);
```

La oveja comiendo se logró separando la cabeza del cuerpo por zonas del modelo, para poder bajarla al comer:

```
if (ovejaComiendo)
{
    ovejaTiempo += dt;
    float p = ovejaTiempo / ovejaDuracion;
    if (p >= 1.0f)
    {
        ovejaComiendo = false;
        ovejaTiempo = 0.0f;
        ovejaAngulo = 0.0f;
    }
    else
    {
        float b = 1.0f;
        if (p < 0.15f) b = p / 0.15f;
        else if (p > 0.85f) b = (1.0f - p) / 0.15f;
        b = b * b * (3.0f - 2.0f * b);
        float masticar = (p > 0.15f && p < 0.85f)
            ? 4.0f * std::fabs(std::sin(ovejaTiempo * 10.0f)) : 0.0f;
        ovejaAngulo = (ovejaAnguloMax - masticar) * b;
    }
}
```



Figura 7. Escena final con los modelos integrados: casa, bandera, personajes, caballo, oveja, cactus y árboles.

4.- Interacción con iluminación (spotlights sobre Steve)

El personaje iluminado es **Steve**, ubicado cerca de la posición (2, 0, 0) de la escena. Se colocaron dos spotlights, uno de color rojo y otro de color azul, a ambos lados de Steve (1.4 unidades a la izquierda/derecha, 1.3 hacia arriba y 0.8 al frente), ambos apuntando hacia él. Como la posición de las luces se calcula a partir de la posición de Steve, lo siguen cuando camina. Cada luz tiene un cono interior de 12.5° y uno exterior de 20°.

```
glm::vec3 objetivoSteve(2.0f + stevePosX,
    alturaTerreno(2.0f, stevePosZ) + 0.8f + stevePosY,
    stevePosZ);
glm::vec3 posicionLuzRoja = objetivoSteve + glm::vec3(-1.4f, 1.3f, 0.8f);
glm::vec3 posicionLuzAzul = objetivoSteve + glm::vec3(1.4f, 1.3f, 0.8f);
glm::vec3 direccionLuzRoja = glm::normalize(objetivoSteve - posicionLuzRoja);
glm::vec3 direccionLuzAzul = glm::normalize(objetivoSteve - posicionLuzAzul);
```

Cada luz se envía al arreglo *spotLights* del shader de iluminación, que incluye el campo *activo* para encenderla o apagarla:

```
glUniform3f(glGetUniformLocation(prog, "spotLights[0].diffuse"), 1.0f, 0.0f, 0.0f);
// ...
glUniform1i(glGetUniformLocation(prog, "spotLights[0].activo"), luzRojaActiva ? 1 : 0);
// ...
glUniform3f(glGetUniformLocation(prog, "spotLights[1].diffuse"), 0.0f, 0.0f, 1.0f);
// ...
glUniform1i(glGetUniformLocation(prog, "spotLights[1].activo"), luzAzulActiva ? 1 : 0);
```

Control: la tecla R enciende o apaga la luz roja y la tecla N enciende o apaga la luz azul. Son independientes, por lo que se pueden ver ambas, solo una o ninguna, en tiempo real. Al iniciar el programa ambas luces están encendidas.

```
// Luces roja (R) y azul (N) sobre Steve
if (key == GLFW_KEY_R && action == GLFW_PRESS)
{
    luzRojaActiva = !luzRojaActiva;
    std::cout << (luzRojaActiva ? "Luz roja activada" : "Luz roja desactivada") << std::endl;
}
if (key == GLFW_KEY_N && action == GLFW_PRESS)
{
    luzAzulActiva = !luzAzulActiva;
    std::cout << (luzAzulActiva ? "Luz azul activada" : "Luz azul desactivada") << std::endl;
}
```

5.- Piso

El piso se carga desde el modelo *Piso.fbx* con sus UVs y texturas. Como el modelo viene con el eje Z hacia arriba (convención de Blender), se gira −90° en X al dibujarlo. Además, el programa lee los triángulos del piso para calcular la altura del terreno en cualquier punto (*alturaTerreno*); con esa altura se apoyan los árboles, casas, animales y personajes, de modo que ninguno queda enterrado ni flotando.

6.- Acomodo de árboles

El modelo del árbol se carga una sola vez y se dibuja en nueve posiciones (los 5 árboles existentes y 4 nuevos). La coordenada Y de cada árbol es la altura del piso en ese punto, con lo cual no atraviesan el suelo ni quedan en el aire:

```
glm::vec3 posicionesArboles[] = {
    {-14.00f, 0.06985f, -8.00f},
    {-7.00f, 0.06094f, -13.00f},
    {-1.00f, -0.00042f, -15.00f},
    {10.00f, -0.00041f, -12.00f},
    {-15.00f, 2.00230f, 0.00f},
    {-12.00f, 3.58176f, 7.00f},
    {-5.00f, 2.37925f, 10.00f},
    {1.00f, 1.00695f, 11.00f},
    {11.00f, 0.00537f, 9.00f}
};
// ...
// Incorporación de árboles.
for (const auto& posicion : posicionesArboles)
{
    model = glm::translate(glm::mat4(1.0f), posicion);
    model = glm::scale(model, glm::vec3(2.0f));
    glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    tree.Draw(lightingShader);
```

Controles del Escenario

Tecla	Función
W A S D / Flechas	Mover la cámara
Mouse	Girar la cámara
Esc	Cerrar el programa
L	Cambia la luz direccional entre blanco y naranja
R	Enciende / apaga la luz roja sobre Steve
N	Enciende / apaga la luz azul sobre Steve
Z	El zorro camina / se detiene
X	El zorro salta
B	Steve camina (animación por huesos)
G	Los habitantes golpean con el hacha
H	El caballo relincha
O	La oveja come pasto
C	Pausa la caminata del Creeper
V	Reanuda la caminata del Creeper
E	El Creeper explota y regresa a su lugar de origen

Problemas encontrados durante el examen

Lo más difícil fue lograr que los modelos quedaran bien apoyados sobre un piso con lomas: al principio algunos quedaban enterrados o flotando, por lo que se creó la función *alturaTerreno*, que obtiene la altura exacta del piso en cada punto. También fue complicado animar la oveja y el caballo, porque son modelos de una sola pieza: para la oveja fue necesario separar la cabeza por zonas del modelo y para el caballo calcular su eje de inclinación. Otro

reto fue lograr que el hacha y el sombrero siguieran los huesos de Steve durante su animación. Por último, al cambiar el color de la luz direccional los objetos se veían afectados, por lo que se ajustó la iluminación de los objetos.

Complejidad y explicación de los ejercicios

El examen combinó iluminación, modelos, animaciones y control por teclado en una sola escena. Lo más complejo fue coordinar todas las animaciones (por código, procedurales y por huesos) sin que se estorbaran entre sí ni chocaran con los objetos. La jerarquía del zorro y las luces que siguen a Steve ayudaron a entender cómo se encadenan transformaciones en OpenGL.

Comentarios generales

Pude integrar una escena con temática de la Revolución Mexicana: un pueblo con casa, iglesia, pozo, molino, bandera, corral y animales, con personajes y animaciones que se controlan desde el teclado. La luz de la escena se puede cambiar entre blanco y naranja, y Steve se ilumina con luces de colores que se encienden y apagan de forma independiente.

Conclusión

Con este examen se consolidaron los conceptos de iluminación, carga de modelos, transformaciones jerárquicas y animación en OpenGL. Crear parte de los modelos en Blender permitió entender todo el proceso, desde el modelado y las texturas hasta su integración en la escena, y programar las animaciones por código dio más control sobre el comportamiento de cada elemento.

Referencias

Blender Foundation. (2026). *Blender* (Versión 5.0.1) [Software]. https://www.blender.org/

Joserapath23. (s. f.). *Extraordinario2027-1* [Repositorio de código]. GitHub. https://github.com/joserapath23/Extraordinario2027-1

de Vries, J. (s. f.). *LearnOpenGL*. https://learnopengl.com/

The Khronos Group. (s. f.). *OpenGL documentation*. https://www.khronos.org/opengl/

Sketchfab. (s. f.). *Modelo 3D de oveja* [Modelo 3D]. https://skfb.ly/6QUXD

Sketchfab. (s. f.). *Modelo 3D de cabra* [Modelo 3D]. https://skfb.ly/oxO7Z

Sketchfab. (s. f.). *Modelo 3D de caballo* [Modelo 3D]. https://skfb.ly/6RxOz

Los modelos de la oveja, la cabra y el caballo se descargaron de Sketchfab y se usan según la licencia indicada por su autor en la página de cada modelo. Los demás modelos de la escena (casa, iglesia, barril, bandera, cactus, aspas, molino, pozo, fogata, sombrero, cerca y persona) fueron elaborados por el alumno en Blender. El piso, el árbol, el zorro, Steve y el Creeper provienen del proyecto base de la asignatura.